



WEB DEVELOPMENT MODULE

Problem Statement Book

Organized By:

**Software Society, Military College of Signals
Campus of NUST**

Date: Tuesday, 10th February 2026 | **Time:** 11:00 hrs – 18:00 hrs

Venue: Library, Military College of Signals | **Total Points:** 100 per Problem

Muhammad Ahmad Sultan

Head, Web Development Module
Vice President, Software Society

Samar Qaiser

Head, Web Development Module
Secretary Events, Software Society

“Crying is allowed. Debugging is mandatory. AI chatbots are your new colleagues. Sleep is optional, deadlines are not. Coffee or tea is essential, but your bank account disagrees. And remember: if your code works on the first try, you're probably not doing it right.

Good luck, brave developers!”

Problem Statement 1: Smart Event Management & Networking Platform

Overview

Build an intelligent event management platform enabling organizers to create, manage, and optimize conferences with AI-powered networking, interactive venue management, and real-time session coordination.

Problem Context

Event organizers struggle with attendee engagement, session capacity management, and facilitating meaningful networking. This platform revolutionizes event management by combining traditional scheduling with intelligent matching algorithms and interactive venue management.

Core Requirements (75 Points)

1. Multi-Tenant Architecture & Event Management (10 Points)

Multi-tenant system where organizers manage multiple events independently with role-based access control (Super Admin, Organizer, Staff, Attendee). Event creation with metadata, duplication/templates, and dashboard showing registrations, revenue, and attendance trends.

2. Advanced Registration System (13 Points)

Conditional registration forms with dynamic fields, early bird pricing with automatic tier transitions, group discounts, multiple ticket types, payment gateway integration (Stripe/PayPal sandbox), QR code confirmation emails, waitlist management, and CSV/Excel export.

3. Interactive Venue Floor Plan Editor (10 Points)

SVG-based floor plan builder with drag-and-drop elements. Upload custom layouts or build from scratch. Define zones, seat/booth assignment with visual representation, real-time capacity indicators, click-to-view details, and responsive viewer for attendees.

4. Progressive Web Application (PWA) Features (10 Points)

Full PWA with service workers and offline capability. QR code check-in system working offline with sync. Install prompt, offline agenda access, background sync for check-ins, push notifications for session reminders.

5. Advanced Search & Discovery (8 Points)

Multi-faceted search with filters for attendees (name, company, industry, interests, availability), sessions (topic, speaker, time, capacity), and exhibitors. Auto-suggest with debouncing, search highlighting, saved preferences, and recent searches.

6. Real-Time Session & Capacity Management (10 Points)

Drag-and-drop session scheduler with automatic conflict detection (venue double-booking, speaker conflicts). Real-time capacity tracking with waitlist, session overflow management, visual indicators (full/available), WebSocket live updates, cancellation handling with notifications.

7. Interactive Event Features (8 Points)

Live polling during sessions with real-time results, Q&A management (submit, upvote, moderator approval), business card scanner using device camera with OCR, digital business card exchange with QR codes, calendar integration (.ics file generation), session notes.

8. Custom Badge Designer (6 Points)

Drag-and-drop badge template builder with dynamic field insertion (name, company, QR code), print preview with multiple badges per page, PDF export, pre-designed templates library.

9. Speech Navigation & Accessibility (Required)

Voice commands for navigation ("Show my agenda", "Find sessions on AI"), text-to-speech for announcements, speech-to-text for Q&A, ARIA labels, keyboard navigation, screen reader compatibility.

10. Gesture Recognition & Motion Control (Required)

WebRTC-based hand gesture recognition for hands-free navigation. Use TensorFlow.js HandPose or MediaPipe Hands. Gestures: swipe to navigate, pinch to zoom. Real-time detection <100ms latency with camera fallback.

AI/ML Integration (20 Points)

1. Intelligent Attendee Matching (8 Points)

ML-based networking recommendations using collaborative filtering. Profile similarity analysis (interests, industry, goals), network graph visualization, "People you should meet" suggestions with conversation starters.

2. RAG-Powered Event Chatbot (6 Points)

Retrieval-Augmented Generation chatbot trained on event data. Answer questions about schedule, speakers, venue, logistics. Context-aware multi-turn conversations. Use open-source models (Transformers.js, TensorFlow.js).

3. Personalized Agenda Generation (3 Points)

AI-generated schedules based on attendee profile, interests, registration responses, industry, and role. Suggest optimal session combinations with travel time. Handle conflicts automatically.

4. Session Recommendation Engine (3 Points)

Behavior-based recommendations (bookmarked sessions, attendance history). "Attendees who liked this also attended" suggestions. Trending sessions based on engagement.

Deployment & Production Readiness (5 Points)

Live deployment on Vercel/Netlify/Railway/Render with SSL/HTTPS. Database hosted on cloud (Supabase/MongoDB Atlas/Railway PostgreSQL). Environment configuration (dev/prod). Error monitoring (Sentry). Lighthouse score >85. Code splitting and lazy loading. CI/CD pipeline (GitHub Actions). Health check endpoint. Production logs.

Data Resources (Free/Public)

- **Sample event data:** Generate using Faker.js or Mockaroo
- **Profile images:** Use RandomUser API or UI Avatars
- **Company/industry data:** Use free tier of Clearbit Logo API or static JSON
- **Training data for ML:** Create synthetic attendee profiles with preferences
- **Chatbot knowledge base:** Use event FAQ templates from GitHub repositories

Sample Use Case

EventTech Conference 2025: 500 attendees, 50 speakers, 30 sessions across 3 days, 5 parallel tracks, networking mixer with AI matching, virtual attendance, exhibitor booths with floor plan, real-time polls during keynotes.

Scoring Breakdown

Category	Points
Multi-Tenant & Event Management	10
Registration System	13
Floor Plan Editor	10
PWA Features	10
Advanced Search	8
Session Management	10
Interactive Features	8
Badge Designer	6
AI: Attendee Matching	8
AI: Event Chatbot	6
AI: Personalized Agenda	3
AI: Recommendations	3
Deployment & Production	5
TOTAL	100

Problem Statement 2: Hospital Operating Room Scheduler with Intelligent Conflict Resolution

Overview

Develop an intelligent OR scheduling system optimizing surgical facility utilization while managing complex medical constraints, staff availability, and emergency cases, balancing patient safety, resource optimization, and operational efficiency.

Problem Context

Hospitals face OR scheduling challenges with conflicting constraints: surgeon availability, equipment sterilization, emergency interruptions, staff fatigue, and patient priorities. Poor scheduling causes extended wait times, underutilized facilities, and staff burnout.

Core Requirements (75 Points)

1. Multi-Tenant Hospital Management Architecture (9 Points)

Multi-tenant system supporting multiple hospitals/facilities. Each manages independent ORs (general, cardiac, neuro, orthopedic), surgical staff, equipment inventory, and patient queues. Role-based access: Hospital Admin, OR Manager, Surgeon, Scheduler, Nurse. Cross-facility resource sharing.

2. Comprehensive Surgery Request Management (10 Points)

Surgery request submission with patient demographics, procedure type, complexity level (1-5), priority classification (Emergency, Urgent, Elective), required surgeon specialization, equipment requirements, estimated duration, pre/post-op requirements, anesthesia type. Approval workflow and bulk import capability.

3. Intelligent Constraint Satisfaction Scheduler (16 Points)

Constraint satisfaction algorithm handling hard constraints (surgeon specialization match, OR capabilities match, equipment availability with sterilization, anesthesiologist availability, pre/post-op room availability, no surgeon >12 hours/24 hours, mandatory breaks, equipment sterilization cycles <3 hours apart) and soft constraints (minimize patient wait time, minimize OR idle time, preferred surgeon, batch similar procedures, minimize overtime, respect surgeon preferences). Real-time conflict detection, cascade rescheduling for emergencies, explanations for delays, optimality scoring.

4. Priority Queue System with Dynamic Reordering (10 Points)

Three-tier priority: Emergency (within 2 hours), Urgent (24-48 hours), Elective (within 30 days). Automatic priority escalation based on wait time. Emergency override with automatic rescheduling. Fair bumping algorithm. Waitlist management. Visual priority queue dashboard with aging indicators.

5. Interactive Drag-and-Drop Calendar Interface (12 Points)

Multi-OR Gantt chart timeline (15-minute increments, 24-hour view). Drag-and-drop surgery blocks with real-time conflict warnings (red), soft violations (yellow), automatic snap to valid slots, visual indicators for setup/surgery/cleanup. Color coding: green (scheduled), yellow (soft violations), red (hard violations), blue (emergency), grey (unavailable). Zoom/filter controls by surgeon, procedure, priority.

6. Equipment & Resource Management (7 Points)

Equipment inventory tracking with surgical instrument sets, sterilization status, specialized equipment, location tracking, sterilization cycle scheduling. Conflict detection when surgeries need same equipment. Automatic alternative suggestions. Maintenance scheduling with OR blocking. Utilization reports.

7. Real-Time Notification & Alert System (5 Points)

WebSocket real-time updates for new emergencies, schedule changes, equipment conflicts, staff availability changes. Email/SMS for surgery scheduled/rescheduled, pre-op instructions, reminders. In-app alert center for unresolved conflicts. Escalation alerts for overdue surgeries.

8. Advanced Search, Filtering & Pagination (6 Points)

Complex search across surgeries (patient name, procedure, surgeon, date range, priority), surgeons (specialization, availability, workload), equipment (type, availability, location), ORs (capabilities, availability). Multi-column sorting. Efficient pagination for 10,000+ records. Saved filter presets. CSV/Excel export.

9. Biometric Authentication & Security (Required)

WebAuthn implementation for passwordless login using fingerprint/face recognition. Support platform authenticators (Touch ID, Face ID, Windows Hello). FIDO2 compliance. Session management with JWT tokens. Audit trail for authentication attempts.

10. Offline-First Medical Records Access (Required)

IndexedDB implementation for offline patient data access. Sync when online using Background Sync API. Conflict resolution for offline edits. Encrypted local storage (Web Crypto API). Quota management for medical imaging files.

AI/ML Integration (20 Points)

1. Surgery Duration Prediction Model (12 Points)

ML regression model predicting actual surgery duration based on procedure type/complexity, surgeon's historical performance, patient characteristics (age, BMI, comorbidities, ASA score), time of day (fatigue), day of week, scheduled duration. Use Random Forest/Gradient Boosting/Neural Network. Evaluation: MAE, RMSE. Feature importance analysis. Prediction

confidence intervals. Continuous learning from completed surgeries. Auto-fill predicted duration when scheduling.

2. Schedule Optimization Recommender (5 Points)

ML-based schedule quality scorer predicting disruption probability. Identify high-risk time slots. Suggest optimal surgery sequences. Pattern recognition from successful vs. problematic schedules.

3. Equipment Failure Prediction (3 Points)

Predictive maintenance based on usage history. Predict equipment failures before occurrence. Proactive maintenance scheduling. Reduce unexpected unavailability.

Deployment & Production Readiness (5 Points)

Live deployment with HIPAA-compliant hosting considerations. SSL/HTTPS with TLS 1.3. Database encryption (at rest and in transit). CORS configuration. Rate limiting. Automated daily backups. Monitoring dashboard (uptime, error rates, API response times). Load testing for 100+ concurrent users. Documentation.

Data Resources (Free/Public)

- **Training data for ML:** Use MIMIC-III Surgery dataset (free after registration) or create synthetic surgery records using medical procedure taxonomies
- **Medical procedures:** Use ICD-10 procedure codes (free from CMS)
- **Equipment data:** Create from standard OR equipment catalogs
- **Historical surgery data:** Generate using medical data generators or Synthea Patient Generator
- **Surgeon/staff data:** Use Faker.js with medical specializations

Scoring Breakdown

Category	Points
Multi-Tenant Architecture	9
Surgery Request Management	10
Constraint Satisfaction Scheduler	16
Priority Queue System	10
Drag-and-Drop Calendar	12
Equipment Management	7
Notification System	5
Search & Pagination	6
AI: Duration Prediction	12
AI: Schedule Optimizer	5
AI: Equipment Failure	3
Deployment & Production	5
TOTAL	100

Problem Statement 3: Algorithmic Courtroom Case Scheduling & Justice Optimization System

Overview

Design an intelligent court case scheduling system optimizing judicial resource allocation while maintaining fairness, transparency, and accountability. Balance judge expertise, case complexity, lawyer availability, and public interest while minimizing justice delivery delays.

Problem Context

Judicial systems face massive case backlogs from inefficient scheduling ignoring judge specialization, case interdependencies, witness availability, and fair workload distribution. Manual systems cannot adapt dynamically to new filings, withdrawals, or adjournments, causing cascading delays.

Core Requirements (75 Points)

1. Multi-Tenant Judicial System Architecture (9 Points)

Multi-level hierarchy: Supreme Court/High Court/District Courts (different tenants). Each court manages independent dockets with cross-court case transfers and appeals. Role-based access: Chief Justice/Presiding Judge, Court Administrator/Scheduler, Lawyers/Public Prosecutors, Litigants/Public (read-only). Jurisdiction management: Civil, Criminal, Family, Tax, Constitutional divisions with territorial boundaries.

2. Comprehensive Case Management System (11 Points)

Case filing/registration with metadata (type, jurisdiction, parties, filing date), complexity score (1-10), urgency classification (Habeas Corpus, Bail, Injunction, Regular), public interest score, connected cases, required hearing duration estimate. Case lifecycle tracking: Filed → Admitted → Listed → Hearing → Reserved → Judgment → Archived with status history and adjournment tracking. Document management with upload pleadings/evidence/orders, version control, access control. Party and lawyer management with multiple parties, lawyer associations, firm conflicts, unavailability calendar.

3. Intelligent Constraint-Based Scheduling Engine (18 Points)

Constraint satisfaction and optimization algorithm continuously scheduling/rescheduling cases. Hard constraints: judge expertise matches jurisdiction, lawyer conflict prevention, judge recusal/bias constraints, courtroom availability, minimum 7-day advance listing, maximum 6 hours court time daily, witness availability coordination. Soft constraints: minimize case pendency (older cases prioritized), balance judge workload, reduce lawyer travel time (batch cases), maximize public interest case visibility (morning slots), minimize total delay days, prefer continuous hearings. Dynamic rescheduling triggers: new urgent case filed, case withdrawn/settled, judge illness/leave, lawyer adjournment request, witness unavailability, courtroom unavailable. Explainability: human-readable delay explanations, conflict graph for unavailability, alternative scenarios, justified priority ordering.

4. Judge Expertise & Workload Management (10 Points)

Judge profiles with specializations (civil, criminal, tax, constitutional, family), experience level, historical case disposal rate, current workload, leave calendar, performance metrics. Bias/recusal tracking with automated conflict detection, family relationship checks, financial interest conflicts. Workload balancing preventing $>2\times$ average pending cases, distribute complex cases evenly, new judges get simpler cases initially.

5. Interactive Court Calendar Heatmap (10 Points)

Visual calendar interface: month view with color-coded case density, week view with detailed time slots, day view with courtroom-wise schedule. Heatmap indicators: green (light workload $<50\%$ capacity), yellow (moderate 50-80%), red (overloaded $>80\%$), purple (blocked). Interactive: click date to see cases, drag-and-drop rescheduling with conflict warnings, filter by case type/judge/lawyer/urgency, quick view upcoming hearings (next 7 days).

6. Conflict Visualization & Resolution Panel (10 Points)

Conflict graph visualization with nodes (Cases, Judges, Lawyers, Courtrooms) and edges (Conflicts-red, Dependencies-blue). Interactive graph to explore conflict chains. Conflict types: lawyer double-booking, judge expertise mismatch, courtroom unavailability, witness scheduling conflicts. Automated resolution suggestions: "Move Case A to 2 PM", "Reassign to Judge B", "Split multi-day hearing". Manual override with justification required.

7. Delay Justification & Transparency Dashboard (7 Points)

Public-facing transparency panel: search any case number for scheduling history, view adjournments and reasons, see queue position, projected next hearing date with confidence interval. Delay explanation engine: "Your case is delayed because: 15 older urgent cases ahead, required judge on medical leave, co-defendant's lawyer unavailable". What-if scenarios. Appeals and escalation: flag cases delayed beyond statutory limits, automatic escalation to Chief Justice, citizens request priority review with justification.

8. Digital Signature & Document Verification (Required)

Web Crypto API implementation for digital signatures on court orders. PKI for certificate management. Verify document authenticity using asymmetric encryption. Generate tamper-proof PDFs with embedded signatures. Blockchain-inspired hash chain for document versioning.

9. Semantic Legal Document Search (Required)

NLP-powered semantic search across judgments and case law using TensorFlow.js with sentence embeddings (Universal Sentence Encoder). Find similar cases with different wording. Citation network visualization. Extract legal entities (acts, sections, precedents) using Named Entity Recognition. Full-text search with relevance ranking and snippet highlighting.

AI/ML Integration (20 Points)

1. Case Duration Drift Prediction (10 Points)

ML regression model predicting actual hearing time based on case type/complexity score, number of parties/witnesses, lawyer's historical case duration patterns, judge's average hearing speed, total pleadings/evidence volume (pages), previous adjournments count, time since filing (aging factor). Use Random Forest/XGBRegressor. Evaluation: MAE on test set. Predict number of hearings needed and total time. Continuous learning from closed cases.

2. Case Outcome & Settlement Probability Prediction (5 Points)

Predict likelihood of settlement before trial, plaintiff victory probability, appeal likelihood. Features: case type, jurisdiction, parties, lawyer win rates, judge's historical ruling patterns, similar precedent outcomes. Suggest mediation for high-settlement-probability cases.

3. Judge-Case Matching Optimization (5 Points)

ML-based judge assignment matching judge expertise to case requirements. Consider judge's disposal rate for complexity level. Predict optimal judge for fastest resolution. Features: judge specialization vectors, case requirement vectors, historical judge-case performance. Output: ranked list of best judges for each case.

Deployment & Production Readiness (5 Points)

Government-grade security with penetration testing checklist. Data privacy compliance (GDPR/equivalent). Audit logging with immutable logs. Disaster recovery with documented backup/restore. Auto-scaling for traffic spikes during high-profile cases. CDN integration for static assets. API documentation (Swagger/OpenAPI). 99.9% uptime monitoring with alerting.

Data Resources (Free/Public)

- **Legal case data:** Use Harvard Caselaw Access Project (free API), CourtListener dataset, or create synthetic cases
- **Judge/lawyer data:** Generate using legal professional data templates
- **Legal documents:** Use public domain court judgments from government websites
- **Training data for ML:** Extract from public court records or create synthetic case histories
- **Citation network:** Build from open-access legal databases
- **Legal entities (acts/sections):** Use government legislative databases (free)

Sample Test Scenario

Metro District Court: 15 judges across 5 divisions, 2,000 pending cases, 50 new cases filed weekly, 100+ lawyers practicing, 10 courtrooms with varying capabilities, 20-30 hearings scheduled daily.

Scoring Breakdown

Category	Points
Multi-Tenant Architecture	9
Case Management System	11
Constraint-Based Scheduler	18
Judge & Workload Management	10
Court Calendar Heatmap	10
Conflict Visualization	10
Delay Justification Panel	7
AI: Duration Prediction	10
AI: Settlement Probability	5
AI: Judge-Case Matching	5
Deployment & Production	5
TOTAL	100

Problem Statement 4: Smart City Parking & Traffic Management Hub

Overview

Build an intelligent citywide parking and traffic management system optimizing urban mobility through real-time parking availability, dynamic pricing, violation detection, and predictive traffic flow analysis serving city administration, parking operators, drivers, and enforcement officers.

Problem Context

Urban areas face severe parking shortages and traffic congestion from inefficient parking space utilization and lack of real-time information. Drivers waste time searching for parking (increasing congestion), parking facilities operate below optimal revenue, and enforcement is reactive rather than proactive.

Core Requirements (75 Points)

1. Multi-Tenant Smart City Architecture (9 Points)

Multi-level tenant structure: City Administration (manages all parking zones), Parking Operators (manage individual facilities: malls, airports, street parking), Enforcement Agency (traffic wardens, tow services). Role-based access: City Planner/Traffic Controller, Parking Facility Manager, Enforcement Officer, Driver/Citizen (mobile app). Zone management: Residential, Commercial, Hospital, Event zones with dynamic zone creation for special events and cross-zone parking privileges (resident permits).

2. Real-Time Parking Inventory & Availability System (13 Points)

Parking space catalog with unique identifiers, type (street, multi-story, underground, surface lot), capabilities (EV charging, disabled access, covered, height clearance), base pricing and dynamic pricing rules. Manual occupancy management system where parking operators/attendants update space status via web interface (Available, Occupied, Reserved, Out-of-service), time-series data storage for historical patterns, WebSocket-based live updates to driver app. Entry/exit logging system where parking attendants or automated barriers log vehicle check-in/check-out times through the web portal. Parking facility details: total capacity, current occupancy, available spaces, operating hours, payment methods, amenities (car wash, valet, security), average turnover rate. Multi-level facility visualization with interactive SVG floor plans, color-coded space availability (green/yellow/red), click-to-reserve specific spots.

3. Intelligent Search, Filtering & Routing (10 Points)

Advanced parking search by location (address, landmark, coordinates). Filter by distance from destination, price range (hourly/daily), availability (immediate, within 15 min), facility type/amenities, EV charging. Sort by price, distance, availability probability, rating. Multi-criteria recommendations: "Cheapest nearby" (within 500m), "Guaranteed availability" (pre-reserve), "Fastest to park" (considering traffic). Route integration with directions to

recommended parking, estimated arrival time, walking distance to destination, alternative parking if first choice fills up en route. Pagination for 10,000+ parking spaces citywide.

4. Dynamic Pricing & Reservation System (10 Points)

Demand-based pricing algorithm: $\text{base rate} \times \text{demand multiplier} \times \text{time-of-day factor}$. Surge pricing during high demand (events, rush hour). Discount pricing for off-peak. Early bird discounts (book 2+ hours advance). Flexible pricing rules: first hour flat rate, subsequent hours variable, daily/weekly pass options, resident permits/exemptions. Reservation system: reserve up to 7 days advance, hold reservation 15 minutes after scheduled arrival, cancellation policy with refunds, no-show penalties. Payment integration: Stripe/PayPal sandbox, support hourly/exit-based/subscription payments, manual check-in confirmation by parking attendant or driver self-check-in via app, digital receipts/invoices.

5. Violation Detection & Enforcement Module (10 Points)

Violation types: overstay (parked beyond paid time), no-permit parking in restricted zones, disabled spot misuse, fire lane/no-parking zone violations, payment evasion. Manual violation reporting by enforcement officers through mobile/web app with photo upload capability, overstay alerts when parking time expires based on reservation/payment data, enforcement officer field app for logging violations. Enforcement workflow: generate e-challan with photo evidence/location/timestamp entered by officer, assign to supervisor for verification, send notification to vehicle owner (email/SMS), payment portal for fines, appeal submission/review process. Enforcement officer app features: map view of violations in zone, manual license plate entry with violation details, issue tickets, view violation history. Violation analytics: hotspot heatmap of frequent violations, repeat offender tracking, revenue from fines dashboard.

6. Interactive City-Wide Parking Map (8 Points)

Map-based visualization with Google Maps/Mapbox integration. Markers for all parking facilities color-coded by availability. Cluster markers for dense areas. Real-time availability updates on map (updated when parking operators log changes). Interactive features: click facility marker to view details and reserve, draw route from current location to parking, filter map by parking type/price/amenities, heatmap layer showing parking demand intensity. User location tracking: show nearby parking within configurable radius, "Park here" quick action for closest available spot, navigation to parked car location (save parking spot manually).

7. Driver Dashboard & Mobile PWA (10 Points)

Progressive Web App for drivers with offline access to saved parking locations, push notifications for parking expiry (15 min before), add to home screen installation prompt. Personal parking dashboard: active parking sessions (location, expiry time, remaining time), parking history with receipts, saved favorite parking locations, upcoming reservations. Quick actions: extend parking time remotely, pay parking fee via app, self-check-in/check-out by scanning QR code at parking location or manual confirmation, report parking facility issues. Wallet integration: store payment methods securely, add balance to parking wallet, transaction history.

8. Admin Analytics & Reporting Dashboard (5 Points)

City-wide metrics: total parking spaces vs. occupied across city, revenue generated (hourly breakdown), peak demand hours/zones, average parking duration by zone. Facility performance: occupancy rate trends (daily/weekly/monthly), revenue per facility, turnover rate (spaces filled per day), utilization percentage. Enforcement statistics: violations issued (by type/zone), fine collection rate, repeat offender count. Export capabilities: download reports as PDF/CSV, scheduled email reports. Visualization: charts/graphs for trends, comparative analysis across facilities.

9. License Plate Recognition (OCR) (Required)

Client-side OCR using Tesseract.js or TensorFlow.js for enforcement officers to quickly capture license plate data from photos taken via mobile device camera instead of manual typing. Camera capture and real-time plate detection. Preprocessing: image enhancement, binarization, noise reduction. Character segmentation and recognition. Validation against regional plate formats. Integration with violation detection system to auto-fill license plate field.

AI/ML Integration (20 Points)

1. Parking Demand Prediction (8 Points)

Time-series forecasting model predicting parking demand. Features: historical occupancy data (hourly for past 6 months from logged check-ins/check-outs), day of week/time of day/month, local events (concerts, sports, conferences), weather conditions (rain, snow, temperature), nearby traffic congestion levels, public holiday indicator. Use LSTM/ARIMA/Prophet. Predict demand for next 24-48 hours. Accuracy: MAPE <15%. Update predictions hourly with new data. Use cases: dynamic pricing adjustments, alert drivers to reserve parking before high-demand periods, identify zones needing more parking.

2. Optimal Parking Recommendation Engine (6 Points)

Multi-objective optimization: minimize cost + walking distance + parking search time, maximize availability probability + user preference score. Personalization: learn user preferences (price-sensitive vs. convenience-focused), remember frequently used parking, recommend based on past behavior. Contextual awareness: destination type (shopping → longer parking, office → daily pass), trip purpose (business → closer parking, leisure → cheaper), time constraints (late for meeting → guaranteed availability).

3. Traffic Flow & Congestion Prediction (6 Points)

Predict traffic congestion caused by parking search using historical parking availability patterns. Identify routes with high "cruising for parking" traffic. Correlate parking unavailability with traffic jams. Suggest alternative routes to avoid congestion. Integration: recommend parking on less congested approach routes, alert drivers when approaching high-traffic parking zones.

Deployment & Production Readiness (5 Points)

Multi-region deployment for global city support. Geospatial database optimization (PostGIS indexes, spatial queries). WebSocket connection pooling for 10,000+ concurrent users. Edge caching for parking availability data (Redis/CloudFlare). API rate limiting per user/IP. Real-time monitoring dashboard: parking occupancy, revenue, violations. Mobile-first performance: Lighthouse score >90 on mobile. Progressive enhancement: works without JavaScript for basic search. GDPR compliance: user data handling and privacy policy.

Data Resources (Free/Public)

- **Parking facilities:** Generate realistic data using city Open Data portals (many cities publish parking data)
- **Occupancy patterns:** Use parking occupancy datasets from Kaggle or city data portals
- **Traffic data:** Use OpenStreetMap for road networks, Google Maps Directions API (free tier)
- **Weather data:** Use OpenWeatherMap API (free tier) for historical and forecast data
- **Event data:** Use Eventbrite API (free tier) or create synthetic event calendar
- **Training data for ML:** Use Melbourne Parking Dataset, San Francisco Parking Dataset, or generate synthetic time-series data
- **License plate images:** Use synthetic license plate generators or ANPR datasets

Sample Test Scenario

SmartCity Metro Area: 500 parking facilities (malls, streets, lots), 25,000 individual parking spaces, 10,000+ daily parking sessions, 5 different pricing zones, 500+ violations daily, 3 parking operators with attendant staff, real-time demand fluctuation during rush hours.

Scoring Breakdown

Category	Points
Multi-Tenant Architecture	9
Real-Time Parking System	13
Search, Filtering & Routing	10
Dynamic Pricing & Reservations	10
Violation & Enforcement	10
Interactive Parking Map	8
Driver PWA Dashboard	10
Admin Analytics	5
AI: Demand Prediction	8
AI: Recommendation Engine	6
AI: Traffic Flow Prediction	6
Deployment & Production	5
TOTAL	100

Repository Setup

1. Create a **public GitHub repository** named: `[TeamName]-P[1-4]-DevCon`
 - Example: `TechTitans-P1-DevCon` for Problem 1
 - Example: `CodeNinjas-P3-DevCon` for Problem 3
 2. Initialize repository with **README.md** containing:
 - Team name and member names with roles
 - Problem statement number and title
 - Setup instructions (clone, install dependencies, run locally)
 - Technology stack used (frontend, backend, database, libraries)
 - Features implemented (checklist format)
 - Known issues or limitations
 - AI/ML model details and training approach
 - Admin/test user credentials for judges
-

Rules

RULE OF THUMB: If you can't explain your code to judges, it's a problem.

Original Work Requirements

1. **All code must be written during the competition** (no pre-competition work)
 2. **Team members must actively contribute** (commit history will be reviewed)
 3. **External code snippets must be attributed** (add comments with source)
 4. **Libraries usage is encouraged, but core logic must be original**
 5. **AI assistance is for learning and productivity, not replacement**
-

Judges' Decision is Final

- Scoring decisions made by judges cannot be appealed
 - No arguments or debates after results are announced
 - Focus on learning and improving, not just winning
-

At Deadline

- **STOP CODING** - No more commits after deadline
 - Wait for judges to review
 - Be available for potential demo or questions
-

Good Luck!